Project Concept Proposal: BIL481 Software Engineering

Group Name: Group-ASYM

Group Members:

- Murat Can Işık
- Yunus Emre Biçer
- Ahmet Sefa Efendioğlu
- Arda Çelik

Academic Institution: TOBB University of Economics and Technology

Academic Term: Fall 2026

1. Project Name

Model Context Protocol (MCP) Based Campus Data Orchestrator
(Official Short Title: Campus Data Orchestrator)

2. Short Description

Higher education campuses rely on multiple disparate digital systems for daily academic and campus operations: Student Information Systems (SIS) for lecture schedules, dining portals for daily cafeteria menus, library turnstile portals for live floor occupancy, and transit portals for shuttle ring schedules. Because these services operate across isolated web pages and siloed databases, students and faculty are forced to check multiple platforms manually to make routine decisions.

Campus Data Orchestrator solves this fragmentation by implementing an open-standard **Model Context Protocol (MCP)** middleware that exposes campus services as standardized, dynamic tools to Large Language Models (LLMs) and client applications. Rather than building a fragile monolithic chatbot, our architecture employs a stateless, horizontally scalable engine backed by distributed Redis caching (with *stale-if-error fault tolerance*), API Gateway token termination (preventing token injection attacks), a client-side deterministic intent router for zero-overhead widget queries, and modular connectors with synthetic test stubs. The system delivers unified answers to complex cross-service student queries (e.g., *"What is for lunch today, and can I catch a ring after my 10:30 class?"*) with guaranteed context integrity, sub-250ms tool execution speed, and complete credential privacy.

3. Requirements (Functional & Non-Functional)

The system requirements below reflect architectural refinements, security boundaries, and

fault-tolerance policies adopted by Group-ASYM:

3.1 Functional Requirements

1. **FR-1 (MCP Protocol Compliance & Tool Registry):** The core server must conform to the official Model Context Protocol (MCP) specification over JSON-RPC 2.0 (supporting Streamable HTTP (using SSE for streaming) and standard I/O (stdio)). It must register tools including get_cafeteria_menu, get_library_occupancy, get_shuttle_schedule, and get_course_schedule with strict JSON Schema definitions.
2. **FR-2 (API Gateway Authentication & Token Termination):** User authentication (OAuth2 / Student SSO JWT) must be terminated at the API Gateway. The gateway must sanitize tokens and inject verified claims into internal request headers (e.g., X-Authenticated-Student-Id) before routing to the MCP server. The LLM agent must never receive or pass user tokens.
3. **FR-3 (Distributed Caching & Stale-If-Error Fallback):** The orchestrator must utilize a distributed Redis cache cluster for upstream connector queries. If an external service times out or fails, the server must automatically return the latest cached data flagged with "isStale": true and an ISO timestamp.
4. **FR-4 (Client-Side Deterministic Intent Router):** The web client must include a lightweight rule-based intent router. Single-domain atomic queries (e.g., "cafeteria menu", "next ring schedule") must bypass the LLM and fetch data directly from the orchestrator cache to eliminate LLM token costs and latency. Complex cross-service queries are routed to the LLM agent. Queries that contain conjunctions or match more than one service are always routed to the LLM agent; ambiguous queries default to the LLM agent.
5. **FR-5 (Modular Connectors with Synthetic Test Stubs):** Each campus service connector must implement a unified connector interface paired with mock/synthetic data generators. When campus external endpoints are unreachable or during automated unit/integration tests, connectors must switch to mock modes without code changes.
6. **FR-6 (Contextual Multi-Tool Synthesis):** For compound queries, the AI agent must dynamically generate a multi-step tool execution plan, invoke the appropriate MCP endpoints, and synthesize the structured JSON payloads into a cohesive natural language answer.

3.2 Non-Functional Requirements

7. **NFR-1 (Layered Latency & Decoupled Execution Speed):** Internal orchestrator tool execution and cached data retrieval must resolve in under **250 ms** (P95). Upstream live scrapes must complete in under **800 ms**. Third-party LLM inference latency is decoupled and tracked independently from server-side tool execution.
8. **NFR-2 (Fault Isolation & Circuit Breaking):** Failure of an individual campus data source (e.g., cafeteria portal downtime) must be isolated via circuit-breaker thresholds (3 consecutive failures trigger a 60s cooldown) and must not block concurrent calls to unaffected services.
9. **NFR-3 (Stateless Horizontal Scalability):** The MCP orchestrator instances must remain strictly stateless. Session tracking, rate limiting, and cache stores are managed in Redis and PostgreSQL, allowing seamless multi-container horizontal scaling under peak loads (min. 100 concurrent requests).
10. **NFR-4 (Zero-Trust Security & PII Protection):** Student Personally Identifiable Information (PII) must be scrubbed before sending prompts to external LLM providers. Communication between the API Gateway, MCP servers, and Redis must enforce TLS/mTLS, with rate limits capped at 30 requests/minute per student.

4. Simple Pseudocode for Implementation Parts

The pseudocode below highlights the client intent router, the dispatcher consuming the gateway-injected auth context (X-Authenticated-Student-Id), and the distributed Redis caching mechanism with stale-if-error fallback:

```
// 1. CLIENT-SIDE LOCAL INTENT ROUTER (Direct Widget Bypass)
function routeStudentQuery(query: string): "DIRECT_WIDGET" | "LLM_AGENT"
{
  const q = query.toLowerCase().trim();
  const directPatterns = [
    /^(what'?s for lunch|today'?s menu|yemekte ne var|yemekhane)/i,
    /^(when is the next (ring|shuttle)|ring saatleri|shuttle
schedule)/i,
    /^(library occupancy|kütüphane doluluk)/i
  ];
  const isCompound =
/\b(and|also|then|after|before|ve|ayrıca|sonra)\b|,/.test(q);
  return (!isCompound && directPatterns.some(p => p.test(q))) ?
"DIRECT_WIDGET" : "LLM_AGENT";
}

// 2. CORE MCP DISPATCHER WITH REDIS & STALE-IF-ERROR
```

```
interface AuthContext { studentId?: string; role: "STUDENT" | "FACULTY"
| "GUEST"; }
interface CacheWrapper<T> { payload: T; cachedAt: number; ttlMs:
number; }

class CampusMCPServer {
  private redis: RedisClient;

  public async handleToolCall(name: string, args: any, auth:
AuthContext): Promise<MCPToolResponse> {
    switch (name) {
      case "get_cafeteria_menu": {
        const date = args.date ?? todayISO(); // YYYY-MM-DD
        return this.fetchWithStaleFallback(`cache:dining:${date}`, 3600,
          () => CafeteriaConnector.fetchMenu(args.date));
      }
      case "get_shuttle_schedule":
        return this.fetchWithStaleFallback(`cache:shuttle:${args.route
|| "MAIN_RING"}`, 120,
          () => ShuttleConnector.getDepartures(args.route));
      case "get_library_occupancy":
        return this.fetchWithStaleFallback(`cache:library:${args.floor
|| "ALL"}`, 60,
          () => LibraryConnector.getOccupancy(args.floor));
      case "get_course_schedule": {
        if (!auth.studentId) throw new UnauthorizedError("Authentication
required.");
        const date = args.day ?? todayISO(); // YYYY-MM-DD
        return
this.fetchWithStaleFallback(`cache:schedule:${auth.studentId}:${date}`,
300,
          () => SISConnector.fetchSchedule(auth.studentId!, args.day));
      }
      default:
        throw new ToolNotFoundError(`Unknown tool: ${name}`);
    }
  }

  private async fetchWithStaleFallback<T>(key: string, ttlSec: number,
fetcher: () => Promise<T>): Promise<any> {
    const raw = await this.redis.get(key);
    const cached: CacheWrapper<T> | null = raw ? JSON.parse(raw) : null;
    const now = Date.now();
```

```
    // Fresh Cache hit (< 15ms)
    if (cached && (now - cached.cachedAt) < cached.ttlMs) {
      return { isError: false, content: [{ type: "json", data:
cached.payload, isStale: false }] };
    }

    try {
      const fresh = await fetcher();
      const wrap: CacheWrapper<T> = { payload: fresh, cachedAt: now,
ttlMs: ttlSec * 1000 };
      await this.redis.set(key, JSON.stringify(wrap), "EX", ttlSec * 3);
      return { isError: false, content: [{ type: "json", data: fresh,
isStale: false }] };
    } catch (err) {
      if (cached) { // Stale-if-error fallback
        return { isError: false, content: [{ type: "json", data:
cached.payload, isStale: true,
          timestamp: new Date(cached.cachedAt).toISOString() }] };
      }
      throw err;
    }
  }
}
```

5. Sample Views

The system implements a dual-view paradigm: a conversational interface for multi-domain inquiries and an instant widget dashboard for atomic campus lookups.

5.1 View 1: Conversational Campus Assistant (Student Chat Interface)

Primary Objective: Provides a natural language chat interface capable of executing complex, multi-service campus queries with transparent tool execution feedback.

Header & Navigation:

- Displays title (*"Campus Data Orchestrator"*) alongside an authenticated student badge (e.g., *"Murat Can Işık - ID: 241101"*).

- Includes system connectivity status pill (*"Connected - SSE Active"*) and quick session reset controls.

Message Stream Layout:

- **User Message Bubbles:** Right-aligned conversational queries (e.g., *"What's for lunch*

today, and can I catch a ring after my 10:30 class?").

- **Tool Telemetry Badges:** Collapsible execution chips rendered immediately above agent responses, detailing active tool calls (e.g., [Tool: get_course_schedule | 200 OK - 32ms], [Tool: get_cafeteria_menu | Cache Hit - 10ms], [Tool: get_shuttle_schedule | Cache Hit - 12ms]).
- **Agent Synthesis Cards:** Left-aligned markdown responses structuring aggregated data (e.g., class end time at 12:20 PM matched against the 12:35 PM Main Gate shuttle departure, followed by the complete cafeteria lunch menu with caloric and allergen indicators).

Interactive Quick Prompts & Input Area:

- **Suggestion Chips:** Contextual shortcut buttons placed above input (e.g., *"Library Quiet Desks"*, *"Next Ring Times"*, *"Tomorrow's Exam Schedule"*).
- **Input Area:** Sticky bottom input field with auto-resizing text area, multi-line support, send button, and token counter indicator.

User Interaction & Feedback: Responses stream incrementally via Server-Sent Events (SSE). If a tool encounters an upstream fault, an amber warning badge is displayed with fallback cache details.

5.2 View 2: Live Campus Glance Dashboard (Fast-Access Widgets)

Primary Objective: Provides zero-overhead, direct access to high-frequency campus metrics powered by the client-side Intent Router and Redis cache without initiating LLM inference.

Header & System Telemetry:

- Global timestamp (*"Campus at a Glance - Live Telemetry"*) with one-click manual refresh button.
- Cache efficiency indicator (*"Redis Cache Hit Rate: 94.2%"*) and upstream service health pulse.

Widget Card Grid Layout:

- **Card A - Library Occupancy Gauge:** Real-time circular capacity gauge (e.g., *"Main Library: 72% Occupied"*) accompanied by per-floor breakdowns (Floor 1: 88% Busy; Floor 2 Quiet Zone: 34 seats free; Floor 3 Silent Study: 60 seats free).
- **Card B - Next Campus Shuttles:** Live departure countdowns categorized by route (Route A Metro Ring: 12:35 PM in 19 min, 12:50 PM in 34 min; Route B Dormitory Line: 12:45 PM in 29 min) with walking time buffer warnings.
- **Card C - Today's Cafeteria Menu:** Tabbed view for Lunch (11:30–14:00) and Dinner (17:00–19:30), highlighting daily soup, main dish, vegetarian alternative, bulgur pilaf, and total caloric sum.
- **Card D - Connector Telemetry & Status:** Live latency badges for campus integrations (SIS Connector: 24ms; Dining Scraper: 18ms; Shuttle GPS: 36ms; Library Feed: 31ms).

User Interaction: Tapping any widget opens an expanded historical telemetry modal or prepopulates the conversational assistant with a relevant deep inquiry prompt.

6. Sample Design Information (Architecture, Tech Stack & Data Models)

6.1 System Architecture

The system is built on a decoupled 5-tier architecture strictly isolating identity, reasoning, and campus connectors:

1. **Client Presentation Tier:** Next.js web application hosting the *Deterministic Intent Router*. Directs atomic queries straight to Redis/MCP and complex queries to the AI Agent.
2. **API Gateway & Authentication Tier:** Fastify API Gateway terminating student SSO/JWT sessions, enforcing rate limits, and injecting verified identity headers (X-Authenticated-Student-Id).
3. **AI Agent & MCP Client Tier:** Manages prompt context and leverages the MCP Client SDK to autonomously negotiate JSON Schema tool executions without holding student credentials.
4. **MCP Server Core (Middleware):** Stateless Node.js/TypeScript server managing tool registration, Redis caching, and circuit-breaker logic over JSON-RPC.
5. **Connector Integration Tier:** Pluggable modular connectors interfacing with campus SIS, cafeteria CMS, shuttle GPS, and library portals (including synthetic test stubs).

6.2 Tech Stack

Layer	Technologies	Description & Role
Backend Core	TypeScript, Node.js, Fastify, @modelcontextprotocol/sdk	Type-safe MCP server handling JSON-RPC tool dispatching and circuit breaking.
Distributed Caching	Redis (Cluster Mode)	Sub-15ms shared TTL caching, distributed rate limits, and stale-fallback storage.
Frontend & UI	React, Next.js, Tailwind CSS, Lucide Icons	Server-side rendered dashboard widgets and streaming SSE chat interface.
AI & Agent Layer	OpenAI / Anthropic Claude /	Multi-intent query parsing and

Layer	Technologies	Description & Role
	Gemini API via MCP Client SDK	dynamic function parameter binding.
API Gateway / Auth	Fastify API Gateway, Jose (JWT verification)	Terminates student OAuth2 tokens; enforces zero-trust boundaries before MCP.
Data Persistence	PostgreSQL, Prisma ORM	Stores persistent student preferences, telemetry logs, and audit trails.
Testing & Mocking	Jest, Supertest, Synthetic Data Generators	Provides deterministic mock adapters for CI/CD pipelines without upstream dependencies.

6.3 Data Models

```
// Core Domain Models & Schemas
interface CafeteriaDailyMenu {
  date: string; // YYYY-MM-DD
  lunch: { soup: string; mainDish: string; vegOption: string; sideDish: string; calories: number; allergens: string[]; };
  dinner: { soup: string; mainDish: string; sideDish: string; calories: number; };
}

interface LibraryOccupancyReport {
  timestamp: string;
  isSimulated: boolean; // True if served by synthetic test stub
  totalCapacity: number;
  totalOccupied: number;
  floors: Array<{ floorId: string; name: string; occupiedSeats: number; maxSeats: number; isQuietArea: boolean; }>;
}

interface ShuttleRouteData {
  routeId: string;
  routeName: string;
  nextDepartures: string[]; // ["12:35", "12:50", "13:05"]
  activeVehicles: number;
  isStale: boolean;
```

```
  lastUpdated: string;
}

interface StudentClassEntry {
  courseCode: string; // e.g., "BIL481"
  courseTitle: string;
  instructor: string;
  classroom: string;
  day: string;
  startTime: string; // "10:30"
  endTime: string;   // "12:20"
}
```

7. Scenarios (User Workflows, Preconditions, & Postconditions)

7.1 Scenario 1: Multi-Service Routine Planning via LLM Chaining

- **Preconditions:**
  - Student is authenticated on the web client with an active SSO token.
  - The API Gateway has validated the token and forwarded the verified X-Authenticated-Student-Id: 241101 header.
  - Connectors for SIS, Cafeteria, and Shuttles are operational.
- **User Workflow:**
  1. Student enters a compound inquiry: *"What's for lunch today, and can I catch a ring right after my 10:30 class finishes?"*
  2. The client Local Intent Router detects a multi-domain query and passes it to the AI Agent.
  3. The LLM plans and issues three tool calls: get_course_schedule(day: "TODAY"), get_cafeteria_menu(date: "TODAY"), and get_shuttle_schedule(route: "MAIN_RING").
  4. The MCP Server receives calls, reads the student ID from the gateway header, queries Redis/connectors, and returns structured JSON responses in parallel (total tool time < 50 ms).
  5. The LLM synthesizes the answer: identifies BIL481 ends at 12:20 PM, recommends the 12:35 PM shuttle from Main Gate, and details the lunch menu.
- **Postconditions:**
  - The student receives a consolidated, multi-service response in a single conversational turn.
  - No student credentials or tokens are leaked to the LLM; telemetry metrics are logged

to PostgreSQL.

7.2 Scenario 2: Direct Widget Access with Stale-If-Error Fallback

- **Preconditions:**
  - The university dining portal is temporarily down due to an upstream network timeout.
  - A cached meal plan from about 2 hours ago (older than the 1-hour TTL, but still retained in Redis) exists in the distributed Redis cluster.
  - The dining connector has already failed twice consecutively, so this timeout is the third consecutive failure.
- **User Workflow:**
  1. Student clicks the *"Today's Menu"* quick widget on the web client.
  2. The Local Intent Router identifies the atomic query and invokes /api/tools/get_cafeteria_menu directly, bypassing the LLM.
  3. The MCP server attempts an upstream fetch, which times out at 800 ms. This third consecutive failure opens the circuit breaker for 60 seconds (NFR-2).
  4. The *stale-if-error* fallback triggers immediately: the server returns the cached Redis data with "isStale": true and original timestamp.
  5. The widget displays the menu with a notice: *"Serving cached menu (Upstream portal currently unavailable)"*.
- **Postconditions:**
  - The student views the dining menu despite the external outage, with zero LLM API cost.
  - Circuit breaker records the fault; subsequent calls within the 60s cooldown window serve cached data in < 15 ms.